САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчёт

Лабораторная работа № 4
Развёртывание приложения на удалённом сервере

Выполнил:

Ермаков Максим Олегович

Группа K3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В лабораторной работе требовалось подготовить удалённый сервер и установить зависимости, развернуть приложение, а затем настроить Nginx как обратный прокси для доступа к приложению извне.

Ход работы

Для развёртывания я использовал приложение бронирования столиков из ЛР3: API Gateway, identity-service, catalog-service, menu-service, reservation-service, review-service, PostgreSQL и RabbitMQ. Работа на удаленном сервере с Ubuntu 24.04 по адресу 31.77.133.132.

Ниже приведены фактически выполненные команды и снимки их результатов.

1 Подключение и исходное состояние сервера

Я использовал подключение it-garage: пользователь root, SSH-порт 22. Перед установкой были проверены пользователь, адрес и состояние сервера.

```
whoami
hostname -I
ls -la /opt /etc/nginx/sites-enabled
dpkg -l docker.io docker-compose-v2 nginx
ss -lnt
```

whoami показывает имя пользователя, hostname -I — адреса сервера. Команда ls проверяет каталоги, dpkg -l — наличие пакетов, ss -lnt — прослушиваемые TCP-порты. Каталог /opt был пуст, Docker и Nginx отсутствовали.

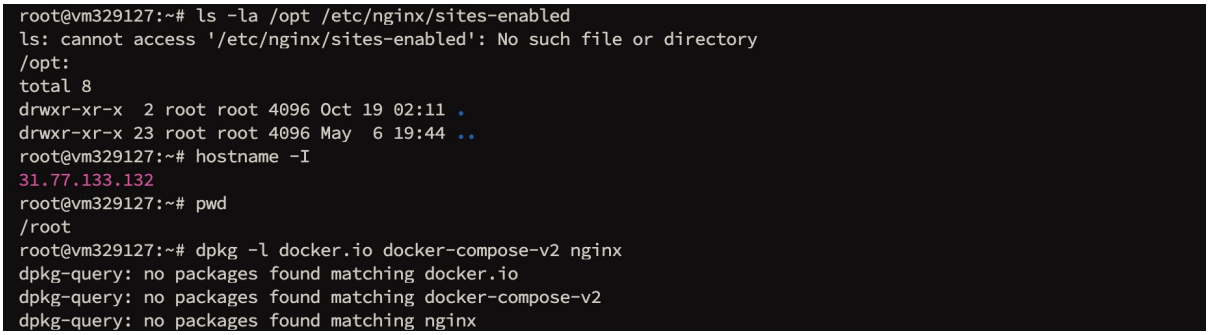


Рисунок 1 — Проверка исходного состояния сервера

2 Установка зависимостей

Для запуска контейнеров потребовались Docker и Docker Compose, а для внешнего доступа — Nginx. Сначала был обновлён список пакетов, затем установлены зависимости. Выполненные команды:

```
apt update
apt install -y docker.io docker-compose-v2 nginx
```

apt update обновляет сведения о доступных пакетах. apt install устанавливает указанные пакеты; параметр -y автоматически подтверждает установку. На рисунках 2 и 3 показаны начало и завершение одной установки.

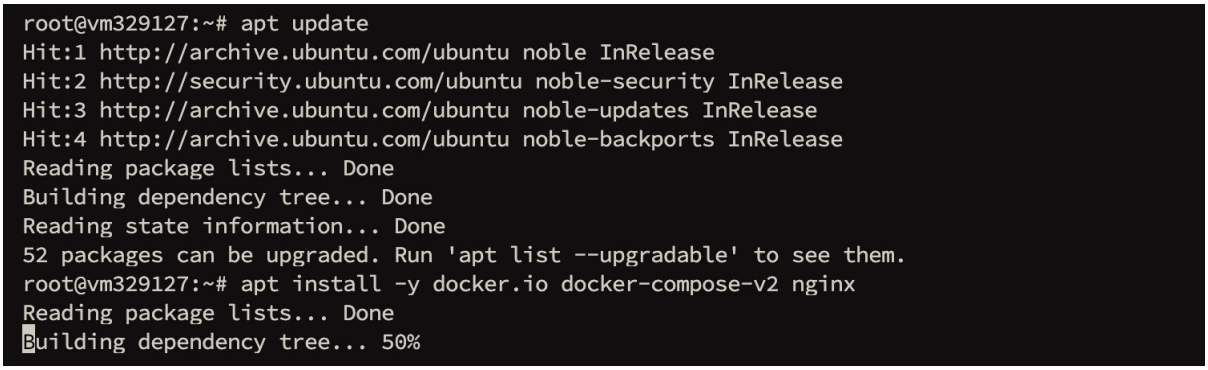


Рисунок 2 — Начало установки зависимостей

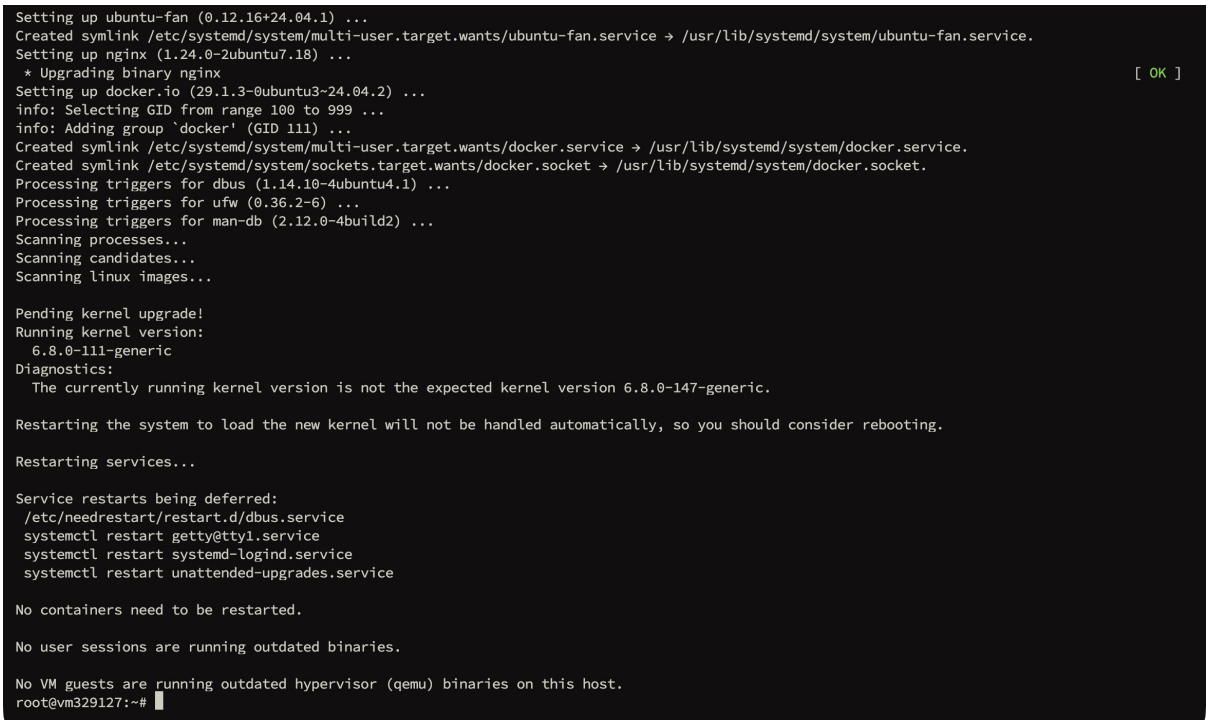


Рисунок 3 — Завершение установки

3 Проверка версий и передача проекта

```
docker --version
docker compose version
nginx -v
docker ps -a
```

Первые три команды показывают версии программ, docker ps -a — все контейнеры. На этом этапе контейнеров ещё не было.

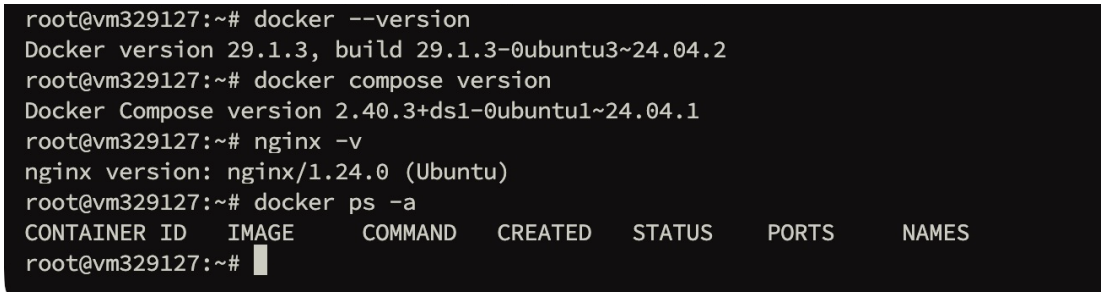


Рисунок 4 — Docker 29.1.3, Compose 2.40.3 и Nginx 1.24.0

Команда docker buildx version выявила отсутствие Buildx. Недостающий компонент сборки был установлен отдельно. Повторная проверка показала версию 0.30.1.

```
apt install -y docker-buildx
```

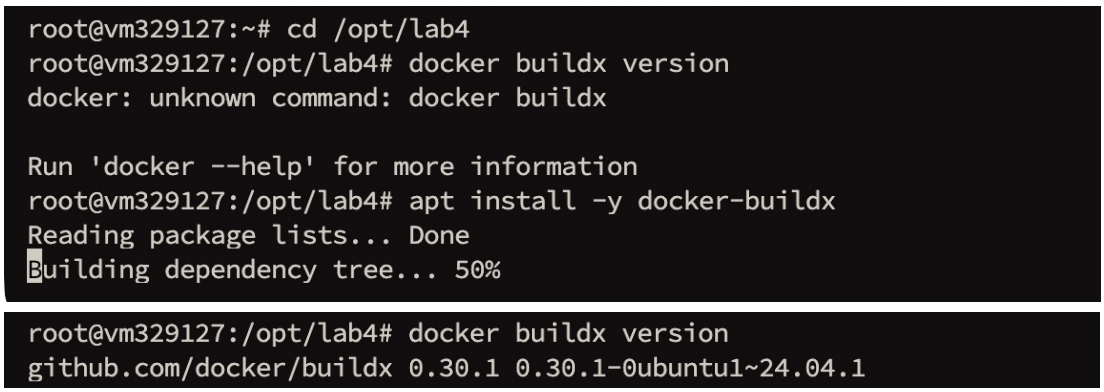


Рисунок 5 — Отсутствующий Buildx и его версия после установки

Для lab4 создана отдельная копия ЛР3 и ветка lab4 от lr3,hw5.

4 Подготовка конфигурации приложения

В новой копии я сохранил исходники сервисов ЛР3. В Docker Compose внутренние сервисы, PostgreSQL и RabbitMQ не публикуют порты на хосте. Только API Gateway доступен на 127.0.0.1:8200, чтобы Nginx мог передавать ему внешние запросы. Данные PostgreSQL и RabbitMQ хранятся в отдельных томах Docker.

```
cd /opt/lab4
ls -la
python3 scripts/create-env.py
docker compose config -q
```

cd переводит оболочку в папку проекта, ls -la показывает файлы, включая скрытые. Небольшой скрипт create-env.py создаёт .env с пятью случайными секретами: паролями PostgreSQL, RabbitMQ и администратора, а также двумя ключами JWT. Файл получает права 600, его значения не выводятся на экран. Повторно существующий файл не перезаписывается.

docker compose config -q проверяет конфигурацию без вывода её содержимого. Это позволяет не показывать секреты. Проверка завершилась без ошибок.

```
root@vm329127:~# cd /opt/lab4
root@vm329127:/opt/lab4# ls -la
total 164
drwxr-xr-x 7 root root   4096 Oct 19 02:55 .
drwxr-xr-x 4 root root   4096 Oct 19 02:55 ..
drwxr-xr-x 2 root root   4096 Oct 19 02:55 docker
-rw-r--r-- 1 root root   3853 Oct 19 02:55 docker-compose.yml
-rw-r--r-- 1 root root     70 Oct 19 02:55 .dockerignore
-rw-r--r-- 1 root root    143 Oct 19 02:55 .env.example
-rw-r--r-- 1 root root     61 Oct 19 02:55 .gitignore
drwxr-xr-x 2 root root   4096 Oct 19 02:55 nginx
-rw-r--r-- 1 root root   1564 Oct 19 02:55 package.json
-rw-r--r-- 1 root root 105713 Oct 19 02:55 package-lock.json
drwxr-xr-x 2 root root   4096 Oct 19 02:55 postman
-rw-r--r-- 1 root root   5310 Oct 19 02:55 README.md
drwxr-xr-x 2 root root   4096 Oct 19 02:55 scripts
drwxr-xr-x 5 root root   4096 Oct 19 02:55 src
-rw-r--r-- 1 root root    369 Oct 19 02:55 tsconfig.json
root@vm329127:/opt/lab4# python3 scripts/create-env.py
Created .env with random secrets; permissions: 600. Values hidden.
root@vm329127:/opt/lab4# docker compose config -q
root@vm329127:/opt/lab4#
```

Рисунок 6 — Файлы проекта, создание секретов и проверка Compose

5 Сборка и запуск контейнеров

`docker compose build`

Команда собирает образы шести сервисов по Dockerfile из проекта. Первая сборка завершилась успешно: у всех образов состояние Built.

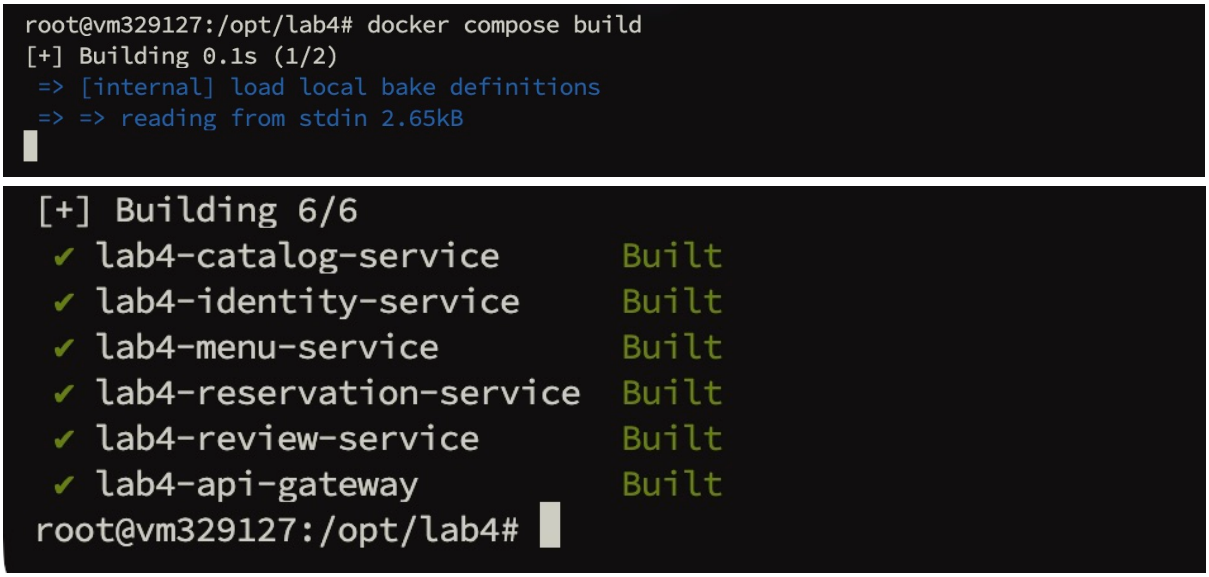


Рисунок 8 — Начало и завершение первой сборки шести образов

`docker compose up -d`

Команда запускает контейнеры в фоновом режиме, загружает PostgreSQL и RabbitMQ, создаёт сеть и тома.

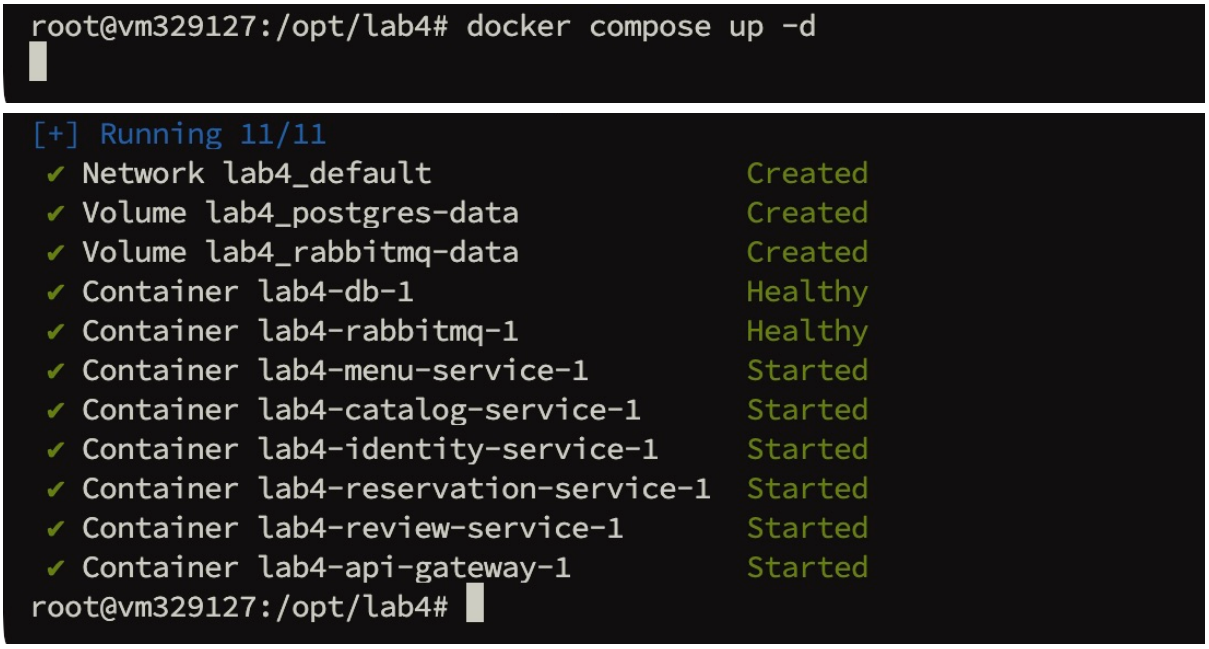


Рисунок 9 — Запуск контейнеров и успешное завершение команды

6 Настройка Nginx как обратного прокси

Я использовал Nginx как единую внешнюю точку входа. Он принимает запросы на порту 80 и направляет их в API Gateway. Был подготовлен файл restaurant-booking.conf

```
cat nginx/restaurant-booking.conf
```

Команда cat показывает содержимое файла. В конфигурации listen 80 задаёт внешний HTTP-порт, server_name — IP сервера, а proxy_pass — адрес API Gateway. Заголовки proxy_set_header передают приложению сведения об исходном запросе.

```
cp nginx/restaurant-booking.conf /etc/nginx/sites-available/restaurant-booking
```

Команда cp копирует конфигурацию в каталог сайтов Nginx.

```
ln -s /etc/nginx/sites-available/restaurant-booking /etc/nginx/sites-enabled/restaurant-booking
```

Команда ln -s создаёт символическую ссылку и включает сайт.

```
nginx -t
systemctl reload nginx
systemctl is-active nginx
```

nginx -t проверяет синтаксис конфигурации. После успешной проверки reload применяет настройку без остановки службы. Команда is-active показывает состояние службы; получен ответ active.

```
root@vm329127:/opt/lab4# cat nginx/restaurant-booking.conf
server {
    listen 80;
    server_name 31.77.133.132;

    location / {
        proxy_pass http://127.0.0.1:8200;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
root@vm329127:/opt/lab4# cp nginx/restaurant-booking.conf /etc/nginx/sites-available/restaurant-booking
root@vm329127:/opt/lab4# ln -s /etc/nginx/sites-available/restaurant-booking /etc/nginx/sites-enabled/restaurant-booking
root@vm329127:/opt/lab4# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@vm329127:/opt/lab4# systemctl reload nginx
root@vm329127:/opt/lab4# systemctl is-active nginx
active
root@vm329127:/opt/lab4#
```

Рисунок 10 — Конфигурация Nginx, её проверка и применение

Для адреса 31.77.133.132 используется подготовленный server_name. Порты внутренних микросервисов, базы данных и RabbitMQ остаются внутри сети Docker.

7 Проверка доступа извне и работы приложения

Для проверки доступа извне запрос выполнен с MacBook в локальной вкладке Termius.

```
curl -i http://31.77.133.132/health
```

curl отправляет HTTP-запрос, -i показывает заголовки. Ответ HTTP 200 OK содержит заголовок nginx и JSON от api-gateway: внешний запрос дошёл до приложения.

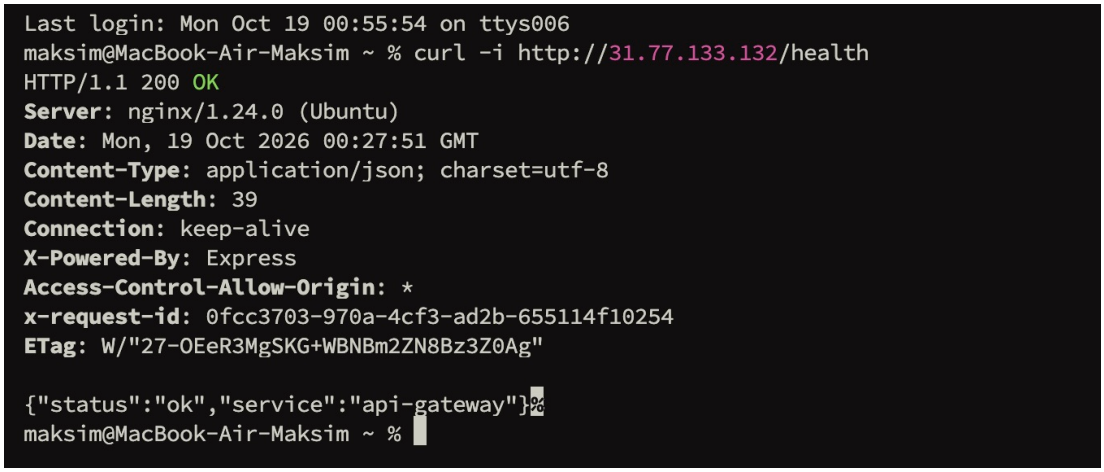


Рисунок 11 — Ответ по публичному IP в локальной вкладке MacBook

Полный сценарий я проверил Node.js-скриптом через публичный адрес Nginx. Пароль берётся из окружения контейнера, JWT хранится в памяти. Секреты в вывод не попадают.

```
docker compose exec -e GATEWAY_URL=http://31.77.133.132/api/v1 api-
gateway node scripts/smoke-test.mjs
```



Рисунок 12 — Успешная проверка полного сценария через Nginx

Сценарий выполнил вход администратора, создание ресторана и связанных данных, регистрацию пользователя, бронь на 26.10.2026, отзыв и подтверждение брони. Все ответы — HTTP 200/201; результат — Smoke test passed. Postman подготовлен, но проверка выполнена этим скриптом.

8 Проверка RabbitMQ и итогового состояния

```
docker compose logs --tail 5 catalog-service
docker compose exec rabbitmq rabbitmqctl list_queues name consumers
messages_ready messages_unacknowledged
```

Первая команда показывает последние строки журнала catalog-service. Вторая — очередь RabbitMQ и число сообщений. В журнале есть обработка review.rating.recalculated для ресторана из сценария; его рейтинг стал avgRating=5, reviewsCount=1.

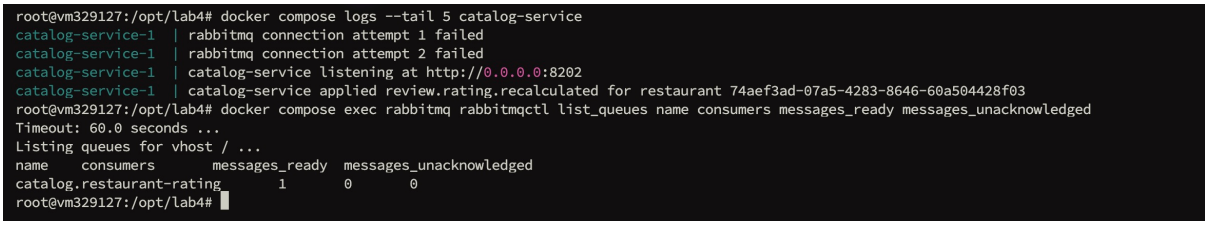


Рисунок 13 — Обработка события рейтинга и состояние очереди RabbitMQ

У очереди один потребитель, ожидающих и неподтверждённых сообщений нет. Доставка подтверждается журналом и рейтингом. Две ошибки подключения при старте устранены встроенным повтором.

```
docker compose ps --format 'table {{.Service}}      {{.Status}}
      {{.Ports}}'
systemctl is-active docker nginx
```

Команды показывают состояния и порты контейнеров, а также состояние служб. Восемь контейнеров работают, PostgreSQL и RabbitMQ — healthy, Docker и Nginx — active.

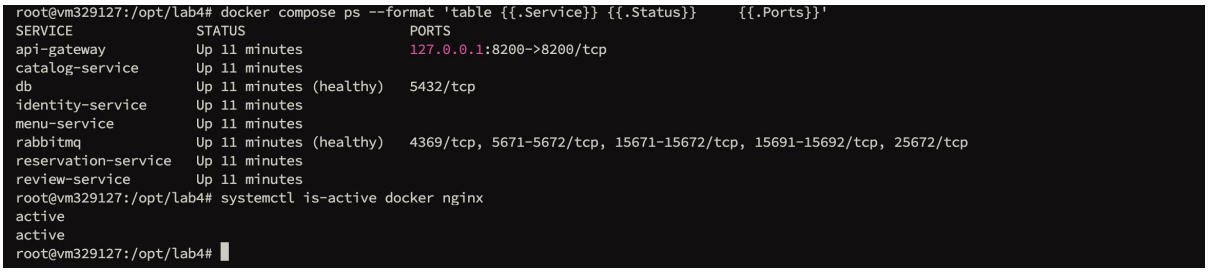


Рисунок 14 — Итоговое состояние контейнеров Docker и Nginx

Вывод

Я развернул приложение через Docker Compose и настроил Nginx как обратный прокси. Установлены зависимости, проверены внешний HTTP-доступ и бизнес-сценарий. Событие RabbitMQ доставлено, рейтинг обновлён. Используется HTTP по IP без TLS.